

# Federated Attention Autoencoders with a Stochastic Aggregation Scheme for Anomaly Detection

Mihailo Ilić\*, Miloš Savić\*, Vladimir Kurbalija\*, Mirjana Ivanović\*, Giancarlo Fortino<sup>†</sup>, Dušan Jakovetić\*

\*Department of Mathematics and Informatics, Faculty of Sciences, University of Novi Sad, Serbia  
 {milic,svc,kurba,mira,dusan.jakovetic}@dmi.uns.ac.rs

<sup>†</sup>Department of Informatics, Modeling, Electronics, and Systems, University of Calabria, Italy  
 giancarlo.fortino@unical.it

**Abstract**—Outlier detection in decentralized data environments is a challenging task for many machine learning implementations, particularly in settings where data cannot be shared. Recently, there have been advances in federated outlier detection, some of which are based on the use of autoencoder networks. The introduction of attention mechanisms to autoencoders boosts their efficiency. However, the application of attention-based models in federated learning remains underdeveloped due to the absence of proper aggregation functions for these types of networks. In our work, we propose two novel aggregation functions tailored for attention-based autoencoders, which better preserve the learned information stored within the memory modules of these networks. We evaluated our approach on the KDDCUP10 dataset, and we showed that the proposed methods achieve up to 2.9% and 5.1% better results for F1 score and AUC ROC respectively when compared to traditional autoencoders.

**Index Terms**—Federated Learning, Representational Learning, Anomaly Detection, Autoencoders

## I. INTRODUCTION

Federated learning (FL) [14] has emerged as a decentralized learning paradigm for training machine learning (ML) models across multiple clients (edge nodes). FL is different from standard distributed learning in that data is never centralized and it is collected at the edge. There are numerous benefits to this approach, some of the most important being increased security and privacy, and the reduction of communication overhead needed to centralize the data or to distribute initially centralized data across a cluster of workers. In most cases, the FL process is coordinated by a central server which conducts client selection and model aggregation [12], [29]. After the clients send their local models to the server, an aggregation step follows. The choice of the aggregation function is extremely important [29], as it directly affects the quality of the federated model. Furthermore, this choice is influenced by challenges like increasing privacy, reducing communication overhead, computational heterogeneity between devices, energy efficiency, and many others [14], [40].

Different model aggregation functions, such as FedAvg [20], FedProx [17], and FedHybrid [25] can yield substantially

different updated global models. The main idea of FL is to train global models which generalize well on the entirety of the distributed dataset. This is one of the main challenges for FL, as no assumptions can be made about the distributions from which the data was drawn at each individual node. Balancing between good generalizability and effectiveness at each edge node is called the model personalization problem. Solutions like client clustering, multi-task learning, and local fine-tuning [2], [10], [15], [35], [42] have been proposed to solve this problem. Approaches like [6], [21] aim to solve the issue of varying data distributions across clients, providing solutions that are distributionally robust.

Representational learning (RL) is a technique which involves learning how to extract relevant features from inputs to ML models during training and inference. By doing so, generalizable features can be obtained from the original input dataset. RL algorithms learn representations that capture underlying factors, allowing this knowledge to be applicable to multiple learning tasks [4]. Learning representations can be done in numerous ways, one of them being through the use of autoencoder networks consisting of 2 parts: the encoder and the decoder. The network learns to represent the inputs in a latent space. Representational learning can be done in an unsupervised, semi-supervised, or self-supervised fashion [4], [7], [34]. Recently, the use of memory-augmented autoencoders, i.e., those relying on attention mechanisms have also been mentioned in the context of federated representational learning [1], [8].

Anomalies or outliers are data instances in a sample that considerably deviate from other members of the sample or from some concept of normality [23], [32]. The detection of anomalies is widely used in a broad variety of applications and they can be classified into three categories: point anomalies, contextual anomalies and collective anomalies. A point anomaly is an individual anomalous data instance (e.g., an illegal transaction, an image of a damaged product in manufacturing). A contextual anomaly is a data instance that is anomalous in a specific time, space, or structural context (e.g., time-series, spatial, spatio-temporal, or graph-based anomalies). A collective anomaly is a set of related, consequent, or dependent data instances that are contextually anomalous. A fundamental assumption in anomaly detection

is that the data space region where the normal data instances exist can be bounded [32].

The main contributions of this paper are:

- an analysis of the challenges involved in aggregating memory modules of attention-based autoencoders in federated learning;
- the proposal of novel FL aggregation functions for attention-based autoencoders;
- an experimental evaluation showing improved performance of said functions compared to using standard aggregation functions like FedAvg [20] or FedProx [17].

The rest of the paper is structured as follows. Section II covers related work on FL, particularly outlier detection and attention-based networks. The approach and novel methods are discussed in Section III. Experimental results are presented and discussed in Section IV. Concluding remarks and plans for future work are given in the last section.

## II. RELATED WORK

Depending on the presence of labels that distinguish normal data instances from anomalous ones in a training dataset, anomaly detection models can be unsupervised, semi-supervised or supervised [23]. The authors of [32] emphasize that there are two big classes of anomaly detection approaches: shallow and deep anomaly detection methods. They can be viewed from a unified perspective, depending on the type of anomaly detection model that belongs to one of the following three categories:

- 1) density estimation and probabilistic models,
- 2) one-class classification models, and
- 3) reconstruction models.

Density estimation and probabilistic models predict anomalies through the estimation of normal data probability distributions. Those methods include classical density estimation, energy-based models, and neural generative models. One-class classification is a discriminative approach to anomaly detection with the goal of learning a decision boundary that separates normal from outlier data instances. Reconstruction models are based on learning low-dimensional data representations of normal data instances. Typical approaches are different variants of principal component analysis, prototypical clustering, and autoencoders.

The work in [27] indicates the main challenges and major issues in contemporary anomaly detection: low anomaly detection recall rate, anomaly detection in high-dimensional data, data-efficient learning of normality, noise-resilient anomaly detection, detection of complex anomalies, and anomaly explanation. The authors also categorized existing deep learning anomaly detection methods into three groups:

- 1) generic normality feature learning (e.g., autoencoders, generative adversarial networks),
- 2) measure-dependent normality feature learning (e.g., clustering, one-class classification), and
- 3) end-to-end anomaly score learning (e.g., anomaly ranking models).

The article also emphasizes self-supervised learning approaches for generic normality feature learning based on the assumption that normal data points are more consistent to self-supervised learning than anomalies.

Unsupervised anomaly detection in federated settings has numerous important applications related to the security and robustness of Internet of Things (IoT) systems, as they generate enormous amounts of data. Consequently, managing these big data systems brings its own challenges in terms of security, ownership, and privacy [3], which FL can help alleviate. The authors of [24] have proposed a self-learning anomaly detection method for intrusion detection in IoT systems. This method uses federated learning to efficiently aggregate anomaly detection profiles. A framework using federated learning to detect malware affecting IoT devices was introduced in [31]. The authors used both unsupervised and supervised anomaly detection models showing that they achieve comparable performance to their centralized counterparts while preserving privacy (due to the privacy-by-design nature of FL). The work presented in [43] investigated federated anomaly detection for finding battery failures in energy storage systems. Besides IoT, federated anomaly detection methods were also applied to sensitive financial data [11]. The authors of [16] addressed the problem of abnormal client detection in horizontal federated learning systems (such clients could be malicious or malfunctioning). Their solution is based on low-dimensional surrogates of model weight vectors that are used to perform anomaly detection at the server side. Adequate client selection may also be carried out through zero-trust approaches like in [39]. In this approach, clients are ranked based on multiple criteria, following which the top  $N$  are selected to participate in learning in each training round. This kind of technique could also be coupled with anomaly detection mechanisms.

Federated representation learning (FRL) joins federated and representational learning [41]. The motivation behind FRL lies in the fact that data distributions between learning clients can differ substantially. Non-IID data can cause substantial issues in federated learning scenarios [9], [22]. Thus, learning a representation of a distributed dataset can increase the performance of collaboratively trained models, including also models for anomaly detection. Having in mind that FL models are typically deep neural networks, since the aggregation of neural network weights is relatively straightforward, autoencoders are the most natural choice for FRL models [26], [28], [45].

Besides providing low-dimensional data representations, autoencoders can also be used for unsupervised anomaly detection following the idea that outliers will have larger autoencoder reconstruction errors than inliers or normal data points [5], [36], [37]. The work in [1] indicated key research directions and empirically investigated various unsupervised anomaly detection approaches in federated settings. In the conducted experiments, the authors evaluated five different models on four different datasets. The examined models include deep autoencoders [5], deep structured energy-based models [44],

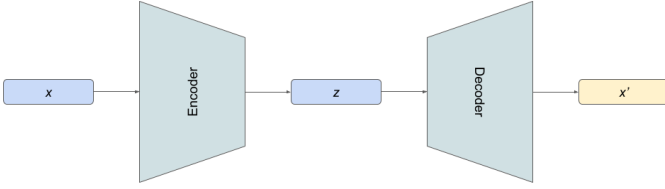


Fig. 1. Diagram of an autoencoder network. The encoder part transforms the input vector  $x$  into a latent representation  $z$ . Following this, the decoder produces  $x'$  from  $z$  in an attempt to reconstruct the input  $x$ .

deep one-class classification [33], neural transformation learning with deep networks [30] and memory-augmented deep autoencoders [8]. These experiments established a benchmark for validating the performance of anomaly detection models in federated settings. As one of the key future research directions, the authors identified the need for more advanced aggregation functions when training federated unsupervised anomaly detection models.

### III. METHODOLOGY

In this paper, we consider outlier detection via autoencoder networks. Autoencoders are general-purpose models, with anomaly detection being only one of their numerous applications. The techniques presented in this paper were validated on this learning task, but can be applied to any other.

These networks consist of two components, an encoder and a decoder. The premise behind this approach is to take an input vector  $x$ , and transform it using the encoder to a different, smaller representation  $z$ . This representation is also referred to as a *code* or *latent representation*. The decoder uses  $z$  as input and outputs  $x'$ , attempting to reconstruct the original input  $x$ . A diagram depicting the high-level structure of an autoencoder network can be seen in Figure 1. The squared  $l_2$  norm difference between  $x$  and  $x'$  is called the reconstruction loss, as seen in (1). This type of network yields low reconstruction loss values for inputs similar to those it was trained on. If the loss is too high, the input vector  $x$  can be considered an outlier.

$$e = \|x - x'\|_2^2 \quad (1)$$

The research presented in this paper focuses on a modified variant of autoencoders, called memory-augmented autoencoders, or MemAE [8]. These autoencoders utilize an attention mechanism, implemented as a memory module between the encoder and decoder parts of the network, a diagram of which can be seen in Figure 2. The memory module is a matrix of size  $N \times C$ , designed to store representative examples of the latent representations of the dataset on which it was trained on.  $N$  represents the number of examples stored in the memory module, while  $C$  is the size of each encoded example. Once encoded into  $z$ , a new data point is compared with all the examples stored in the memory module. A new code  $z'$  is constructed via a linear combination of the representatives most similar to the new data point  $z$  after which  $z'$  is fed into

the decoder. The idea behind this approach is to additionally penalize examples not seen during training, having them produce a larger reconstruction loss. A detailed description of this approach can be found in [8], along with experimental results in centralized learning scenarios.

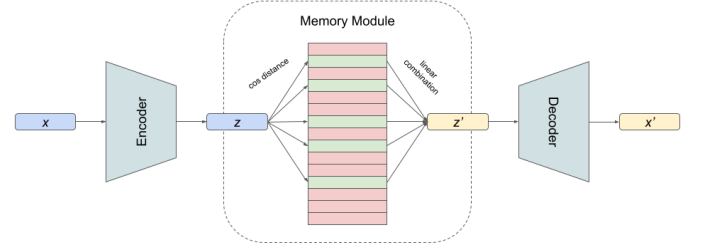


Fig. 2. Diagram of the MemAE model. The encoder outputs a latent representation  $z$  of input  $x$ . This vector, along with the memory module, produce a new representation  $z'$ , which is fed into the decoder to produce the output.

The authors of [1] conducted research on the use of different outlier detection models, one of which was MemAE. In their experiments, the MemAE model performed quite well over four experimental datasets with varying hyperparameters, compared to regular autoencoders or even some other advanced methods. Nevertheless, it is pointed out in [1] that structurally specific elements, like the memory module of MemAE, are not propagated during the aggregation process of FL. The authors argue that this is a limitation to the effective use of this approach, as specific heuristics in the memory module are not aggregated.

In this work, we aim to introduce novel aggregation methods for attention-based autoencoder networks.

#### A. Memory Matrix Aggregations

Neural networks are ML models commonly used in FL settings. Because of this, various aggregation functions, like FedAvg [20] and FedProx [17], have been developed which handle neural network weights received from edge nodes, and aggregate them into the global model weights based on various importance metrics.

The memory module of MemAE is a non-typical element of a neural network. This attention mechanism is a matrix of dimensions  $N \times C$  where  $N$  is the number of example codes the memory module should store, while  $C$  represents the dimension of the encoded vectors. Intuitively, the rows in the matrix store representative results of the training dataset used to tune the model. This matrix is constructed in such a way as to minimize the entropy between the particular vectors stored in its rows. The idea is to obtain a broad set of representative vectors, which can cover the distribution of the training dataset well.

In the experiments conducted in this paper, we propose and validate two new aggregation strategies which take into account both the network weights and the memory module:

- 1) **Naive aggregation** – FedAvg and similar methods apply a variant of weighted averaging over the weights of

the networks obtained from edge nodes. In our first approach, we propagate both the weights and the memory module to the FL coordinator, after which the set of memory modules is treated the same as the set of network weights and is aggregated using a weighted average modified by additional proximal terms via FedProx [17].

- 2) **Random row permutations** – Information is stored in the rows of the matrix of the memory module. In this approach, an aggregated memory module is formed by randomly selecting rows from all the matrices received from the edge nodes during one round of training. The rows that form the newly aggregated memory matrix are taken uniformly at random from the memory modules belonging to the clients, which is illustrated in Figure 3.

The naive aggregation method simply extends regular aggregation methods like FedProx, having them treat the memory module like any other weight in the neural network. Previously in related work [1], the memory module was not propagated during federated training. In our proposed naive aggregation method, the memory module is also shared with the FL coordinator, and is aggregated using the same approach as all other network weights are, i.e. using some sort of weighted average. However, this approach does not take into account that information in these matrices is stored in their rows, one representative of the dataset in each row. Thus, the order of the rows (vectors) in each matrix received from the edge nodes would influence the resulting aggregated matrix. Just by swapping two rows in a single edge matrix, the resulting global matrix would be different. The second approach takes into account the nature of the data being stored in the memory module and aims to preserve the information learned from the edge nodes. The FL server simply selects a random subset of rows from each edge node, as seen in Figure 3, and stacks them to form a new matrix of the same dimensions. This new matrix contains specific encoded vectors learned from the distributed dataset, and is then used as a memory module in the new global model, which is distributed to the edge nodes. The rest of the weights are aggregated using a standard approach, which in our case is FedProx.

### B. The Dataset

The experiments in this paper were conducted on the KDD-CUP10 [38] network intrusion dataset, which is a 10% subset of the original KDDCUP dataset. It contains 494021 data samples, with 42 different continual and categorical attributes, including the class label. The classes cover normal network traffic and 4 categories of network intrusion attacks: denial of service, unauthorized access from a remote machine, unauthorized access to root privileges, and probing or surveillance attacks. Following data preprocessing, the final number of attributes was 115, excluding the class.

All of the network intrusion attacks are merged into a single class, opposed to regular network usage. This gives an outlier ratio of 19.69%.

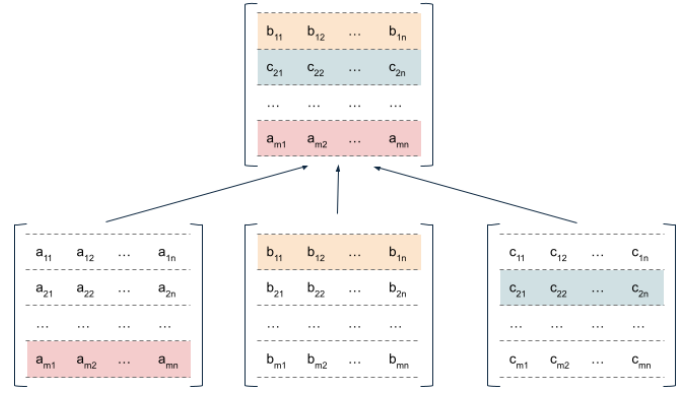


Fig. 3. New global memory matrix aggregation via random selection from matrices submitted by edge nodes during one training round. By preserving the rows containing representative codes, no information stored within them is lost.

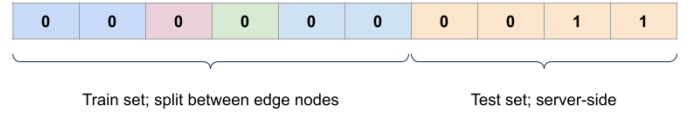


Fig. 4. The test set contains a 50-50 split of inlier (marked "0") vs outlier (marked "1") data points. The leftover datapoints are all inliers, and are distributed randomly between edge nodes. Local datasets are used only for training purposes.

The dataset was split into training and test sets. The test set contains all outlier data points. A random selection of inliers is added to the test set, forming a 50-50 split of inliers and outliers. The remaining inlier data is then split into  $E$  different subsets for training, where  $E$  represents the number of edge nodes in the system. This process can be seen in Figure 4. In our experiments, each of the  $E$  edge nodes contained the same amount of data.

### C. Experimental Setup

Two types of models were used in these experiments: regular autoencoders and memory-augmented autoencoders (MemAE).

The experiment covered multiple scenarios:

- 1) Independent local training – Each of the edge nodes trains its own model, using data available locally. No parameter sharing (FL) is involved in this process. In this scenario, regular autoencoders were trained;
- 2) Autoencoder-based FL – Edge nodes train collaboratively through FL. The model in question is a regular autoencoder;
- 3) MemAE FL with naive aggregation;
- 4) MemAE FL with random row permutations.

The first two experiments are used as baselines to which the newly proposed aggregation strategies were compared. Independent local training serves as the first baseline to verify the motivation for using FL in these scenarios, while the second experiment represents the baseline FL strategy.

In each of the FL experiments, FedProx was used as a base aggregation strategy. This means that the neural network weights for all scenarios, 2 through 4, were aggregated using this aggregation function. In scenario 3, FedProx was also used to aggregate the memory module (naive aggregation), while in scenario 4, the random row permutation strategy was used to aggregate the memory module.

TABLE I  
HYPERPARAMETERS OF THE FL EXPERIMENTS

| Parameter                | Value for $E = 3$ | Value for $E = 50$ |
|--------------------------|-------------------|--------------------|
| Fraction Fit             | 100%              | 10%                |
| Batch Size               | 64                | 1024               |
| # Global Training Rounds | 10                |                    |
| FedProx Proximal $\mu$   | 0.01              |                    |
| # Local Epochs           | 5                 |                    |
| Optimizer                | Adam              |                    |
| Learning Rate            | 0.01              |                    |
| Memory Dimension ( $N$ ) | 50                |                    |
| Encoding Size ( $C$ )    | 10                |                    |

Two sets of experiments were conducted: in the first, the number of edge nodes  $E$  was set to 3, while in the second,  $E = 50$ . All edge nodes participated in the training process in the first set of experiments. During federated training in the second set of experiments, a fraction of edge nodes chosen uniformly at random participated in each global training round, as seen in Table I.

At the end of each global training round, after model aggregation, the newly created global model was validated against the test set on the server side. In this approach, we aim to train a robust global model capable of generalizing well across the distributed environment. In independent local training, each edge node validates its model against the shared test set.

Validation was done by applying min-max normalization on all of the reconstruction losses for data points in the test set. Following this, the threshold value that maximizes the F1 score based on the precision-recall curve was selected, in order to compare the best possible models in each case.

#### IV. RESULTS AND DISCUSSION

In each experiment, all participating edge nodes contributed exactly 50 epochs of training. The federated learning experiments lasted for 10 global training rounds, and regardless of client selection, each edge node trained for 5 epochs in one round. In independent local experiments, all edge nodes trained their own models independently for 50 epochs. After each epoch in local training and each global training step during federated training, the trained model was validated on the centralized test set with a 50-50 inlier-to-outlier ratio. For each experiment, the model with the highest F1 score was taken as the best during that training round, and these are the results shown in Tables II and III. This was done in order to compare the best possible models in each of the scenarios.

Table II depicts the experiments carried out on 3 edge nodes. The row permutation function produced the best aggregated

model in terms of both F1 and AUC ROC scores. This approach yielded an F1 score of 98.2%, which is significantly higher than the regular autoencoder approach, which is 95.3%. The naive aggregation method was also scored higher than the autoencoder, at 97.6%. The local independent models in this experiment significantly underperformed, with the best one achieving an F1 score of only 87.8%.

TABLE II  
MAXIMUM RECORDED METRIC VALUES WHEN  $E = 3$

| Experiment          | Precision | Recall      | F1 Score    | AUC ROC     |
|---------------------|-----------|-------------|-------------|-------------|
| Local Independent   | <b>99</b> | 78.9        | 87.8        | 84.2        |
| Regular Autoencoder | 98.8      | 92.1        | 95.3        | 93.8        |
| Naive Aggregation   | 97.1      | <b>98.2</b> | 97.6        | 97.8        |
| Row Permutation     | 98.5      | 97.8        | <b>98.2</b> | <b>98.9</b> |

Experiments with 50 edge nodes show a significant increase in the performance of local models, which can be seen in Table III. The best local model (out of 50) is tied with the federated model aggregated via the row permutation approach, in terms of F1 measure, at 98.9%. However, the federated model is slightly better in terms of AUC ROC, by a margin of 0.1%. The autoencoder performed worse than the row permutation approach with the MemAE model, with an F1 score of 96.2%. The naive aggregation method drastically underperformed with the highest recorded F1 score of only 85%. This can be attributed to the approach in aggregating the memory matrix, where it is treated like other weights in the network and aggregated in the same way as them. The nature of the data stored within the memory module is not taken into account by this approach, as discussed in Section III-A.

TABLE III  
MAXIMUM RECORDED METRIC VALUES WHEN  $E = 50$

| Experiment          | Precision   | Recall      | F1 Score    | AUC ROC     |
|---------------------|-------------|-------------|-------------|-------------|
| Local Independent   | 98.3        | <b>99.6</b> | <b>98.9</b> | 99.3        |
| Regular Autoencoder | <b>98.6</b> | 93.8        | 96.2        | 97          |
| Naive Aggregation   | 76.7        | 95.4        | 85          | 89.7        |
| Row Permutation     | 98.5        | 99.3        | <b>98.9</b> | <b>99.4</b> |

As seen in Table III, the best independent model achieved the same results as the row permutation based federated network. Since 50 different local models have been trained independently, it is worth exploring the performance of models from individual edge nodes. Table IV shows the aggregated statistics for all 50 edge nodes. The best F1 score is 98.9%, while the worst is 79.7%. In the 50th percentile, the F1 score is at 97.4%, which is still better than the regular autoencoder network trained in a federated fashion. However, the 75th percentile shows that most of the edge nodes trained models which underperformed compared to the row permutation approach. Some of the clients managed to train good models, however, a lot of them would have benefited from federated training, particularly from the row permutation approach. The mean F1 score was 95.3%, which is significantly lower than the row permutation approach, and is also outperformed by the basic federated autoencoder.

TABLE IV  
LOCAL TRAINING STATISTICS E = 50

| Statistic | F1 Score    | AUC ROC     |
|-----------|-------------|-------------|
| Mean      | <b>95.3</b> | <b>95.1</b> |
| Min       | 79.7        | 78.2        |
| 25%       | 94.2        | 93.7        |
| 50%       | 97.4        | 97.9        |
| 75%       | 98.1        | 99.1        |
| Max       | 98.9        | 99.5        |

F1 Scores (E = 3)

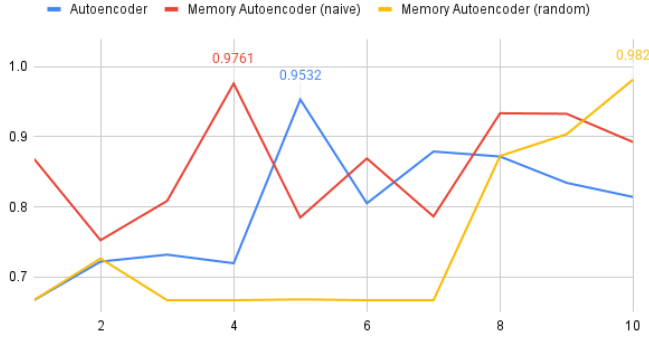


Fig. 5. F1 scores through 10 global rounds of federated training on 3 edge nodes.

Figures 5 and 6 show the changes in F1 and AUC ROC scores during the 10 rounds of federated training on 3 edge nodes. The regular autoencoder and naive aggregation MemAE achieved their best results in the first half of training. The random permutation approach was more stable in that regard, having consistent values in the early stages of training and, by the end, yielding the best performing models.

The training process on 50 edge nodes produced the same end result, with a different training dynamic. Figures 7 and 8 show that random aggregation yielded the best model somewhere in the middle of training. The naive approach dropped off immediately, further emphasizing the importance

ROC AUC Scores (E = 3)

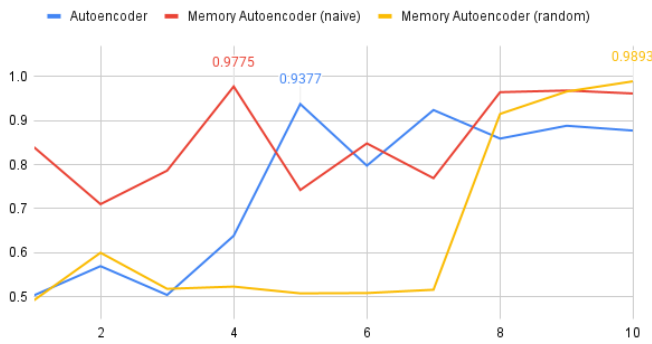


Fig. 6. AUC ROC scores through 10 global rounds of federated training on 3 edge nodes.

F1 Scores (E = 50)

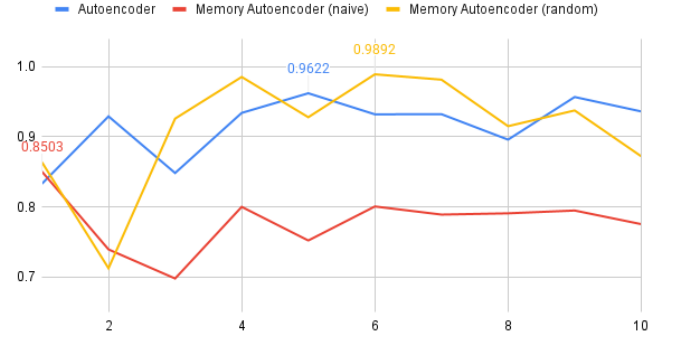


Fig. 7. F1 scores through 10 global rounds of federated training on 50 edge nodes.

ROC AUC Scores (E = 50)

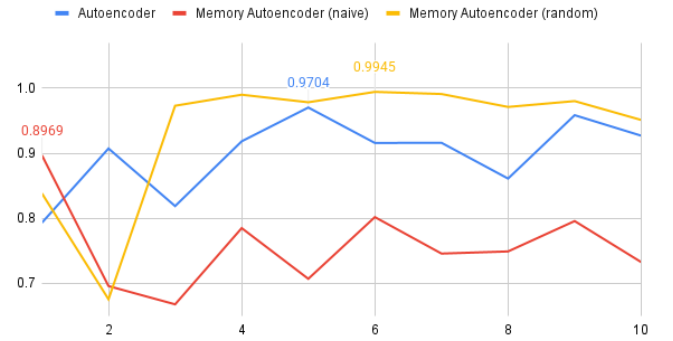


Fig. 8. AUC ROC scores through 10 global rounds of federated training on 50 edge nodes.

of handling the intricacies of the memory module carefully. The regular autoencoder achieved solid results, however, it still underperformed compared to the row permutation approach.

The data shows that it is possible to obtain high quality models by using well-defined aggregation functions that take into account the specifics of the memory-augmented autoencoder. This approach outperformed the standard approach of training regular autoencoders. The naive aggregation function demonstrates the importance of taking into account the heuristics of the attention-based model and the importance of handling the latent representations stored in the memory module with care. The newly proposed aggregation function based on random row permutations showed considerable improvement over other methods. As a stochastic approach, it uniformly samples encoded representations from memory modules provided by clients. The increased performance comes from the preservation of the learned representations that this approach offers. Extending this approach with a guided process of selecting representative examples from the memory modules could further improve its robustness and results. One such way is to make use of submodular optimization [13], [18], [19], and selecting different representatives based on the amount of



new information they carry. This would optimize the process of aggregating the memory matrix.

## V. CONCLUSION

Federated learning is currently generating a lot of interest from both academia and industry. Some work has already been done on the use of different techniques for outlier detection in federated settings. However, the topics of federated representational learning and outlier detection are still fairly unexplored, with room for different research directions.

The results presented in this paper support the notion that row-preserving aggregation functions for attention-based autoencoders yield better results when training in federated environments. Contextual information is stored in the latent representations kept in the memory modules of individual edge nodes, and it is crucial to keep them intact in order to preserve the knowledge gained during training. Information is destroyed when simply propagating the memory module and aggregating it in the same way as neural network weights.

The findings in this paper show promising initial results validated on the KDDCUP dataset. Broader validation is needed with different baseline aggregation algorithms, along with different datasets and ML tasks. Future work also includes the validation of the aggregation techniques proposed in this paper on additional data modalities (i.e., time-series). Furthermore, a more extensive and formal analysis of the random permutation approach is needed, as it is a stochastic process.

We also plan to develop new aggregation techniques based on submodular optimization [13], [18], [19]. This technique helps when selecting examples from a dataset that minimize redundancy with each other, which could lead to a more optimal selection of representative encodings of the federated dataset.

## ACKNOWLEDGMENT

This paper has been supported by the European Union's Horizon Europe research and innovation actions under grant agreement No 101135775 (PANDORA).

The authors from the University of Novi Sad gratefully acknowledge the financial support of the Ministry of Science, Technological Development and Innovation of the Republic of Serbia (Grants No. 451-03-137/2025-03/ 200125 & 451-03-136/2025-03/ 200125).

## REFERENCES

- [1] A. Anwar, B. Moser, D. Herurkar, F. Raue, V. Hegiste, T. Legler, and A. Dengel. Fedad-bench: A unified benchmark for federated unsupervised anomaly detection in tabular data. In *2024 2nd International Conference on Federated Learning Technologies and Applications (FLTA)*, pages 115–122. IEEE, 2024.
- [2] A. Armacki, D. Bajovic, D. Jakovetic, and S. Kar. Personalized federated learning via convex clustering. In *2022 IEEE international smart cities conference (ISC2)*, pages 1–7. IEEE, 2022.
- [3] F. M. Awaysheh, M. Alazab, S. Garg, D. Niyato, and C. V. Verikoukis. Big data resource management & networks: Taxonomy, survey, and future directions. *IEEE Commun. Surv. Tutorials*, 23(4):2098–2130, 2021.
- [4] Y. Bengio, A. Courville, and P. Vincent. Representation learning: A review and new perspectives. *IEEE transactions on pattern analysis and machine intelligence*, 35(8):1798–1828, 2013.
- [5] Z. Chen, C. K. Yeo, B. S. Lee, and C. T. Lau. Autoencoder-based network anomaly detection. In *2018 Wireless Telecommunications Symposium (WTS)*, pages 1–5, 2018.
- [6] Y. Deng, M. M. Kamani, and M. Mahdavi. Distributionally robust federated averaging, 2021.
- [7] A. Dosovitskiy, J. T. Springenberg, M. Riedmiller, and T. Brox. Discriminative unsupervised feature learning with convolutional neural networks. *Advances in neural information processing systems*, 27, 2014.
- [8] D. Gong, L. Liu, V. Le, B. Saha, M. R. Mansour, S. Venkatesh, and A. v. d. Hengel. Memorizing normality to detect anomaly: Memory-augmented deep autoencoder for unsupervised anomaly detection. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 1705–1714, 2019.
- [9] M. Haller, C. Lenz, R. Nachtigall, F. M. Awaysheh, and S. Alawadi. Handling non-iid data in federated learning: An experimental evaluation towards unified metrics. In *2023 IEEE Intl Conf on Dependable, Autonomic and Secure Computing, Intl Conf on Pervasive Intelligence and Computing, Intl Conf on Cloud and Big Data Computing, Intl Conf on Cyber Science and Technology Congress (DASC/PiCom/CBDCCom/CyberSciTech)*, pages 0762–0770, 2023.
- [10] F. Hanzely and P. Richtárik. Federated learning of a mixture of global and local models. *arXiv preprint arXiv:2002.05516*, 2020.
- [11] D. Herurkar, S. Palacio, A. Anwar, J. Hees, and A. Dengel. Fin-Fed-OD: Federated Outlier Detection on Financial Tabular Data, 2024.
- [12] M. Ilić, M. Ivanović, V. Kurbalija, and A. Valachis. Towards optimal learning: Investigating the impact of different model updating strategies in federated learning. *Expert Systems with Applications*, 249:123553, 2024.
- [13] R. K. Iyer and J. A. Bilmes. Submodular optimization with submodular cover and submodular knapsack constraints. *Advances in neural information processing systems*, 26, 2013.
- [14] P. Kairouz, H. B. McMahan, B. Avent, A. Bellet, M. Bennis, A. N. Bhagoji, K. Bonawitz, Z. Charles, G. Cormode, R. Cummings, et al. Advances and open problems in federated learning. *Foundations and trends in machine learning*, 14(1–2):1–210, 2021.
- [15] L. Li, Y. Fan, M. Tse, and K.-Y. Lin. A review of applications in federated learning. *Computers & Industrial Engineering*, 149:106854, 2020.
- [16] S. Li, Y. Cheng, Y. Liu, W. Wang, and T. Chen. Abnormal client behavior detection in federated learning, 2019.
- [17] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith. Federated optimization in heterogeneous networks. *Proceedings of Machine learning and systems*, 2:429–450, 2020.
- [18] H. Lin and J. Bilmes. An application of the submodular principal partition to training data subset selection. In *NIPS workshop on Discrete Optimization in Machine Learning*. Citeseer, 2010.
- [19] H. Lin and J. A. Bilmes. How to select a good training-data subset for transcription: submodular active selection for sequences. In *Interspeech*, pages 2859–2862, 2009.
- [20] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Artificial intelligence and statistics*, pages 1273–1282. PMLR, 2017.
- [21] M. Mohri, G. Sivek, and A. T. Suresh. Agnostic federated learning, 2019.
- [22] L. Nascimento, F. M. Awaysheh, and S. Alawadi. Data skew in federated learning: An experimental evaluation on aggregation algorithms. In *2024 2nd International Conference on Federated Learning Technologies and Applications (FLTA)*, pages 162–170, 2024.
- [23] A. B. Nassif, M. A. Talib, Q. Nasir, and F. M. Dakalbab. Machine Learning for Anomaly Detection: A Systematic Review. *IEEE Access*, 9:78658–78700, 2021.
- [24] T. D. Nguyen, S. Marchal, M. Miettinen, H. Fereidooni, N. Asokan, and A.-R. Sadeghi. D<sup>2</sup>IoT: A Federated Self-learning Anomaly Detection System for IoT. In *2019 IEEE 39th International Conference on Distributed Computing Systems (ICDCS)*, pages 756–767, 2019.
- [25] X. Niu and E. Wei. Fedhybrid: A hybrid federated optimization method for heterogeneous clients. *IEEE Transactions on Signal Processing*, 71:150–163, 2023.
- [26] D. Novoa-Paradela, O. Fontenla-Romero, and B. Guijarro-Berdiñas. Fast deep autoencoder for federated learning. *Pattern Recognition*, 143:109805, 2023.

- [27] G. Pang, C. Shen, L. Cao, and A. V. D. Hengel. Deep Learning for Anomaly Detection: A Review. *ACM Computing Surveys*, 54(2), Mar. 2021.
- [28] M. Polato. Federated variational autoencoder for collaborative filtering. In *2021 International Joint Conference on Neural Networks (IJCNN)*, pages 1–8, 2021.
- [29] P. Qi, D. Chiaro, A. Guzzo, M. Ianni, G. Fortino, and F. Piccialli. Model aggregation techniques in federated learning: A comprehensive survey. *Future Generation Computer Systems*, 150:272–293, 2024.
- [30] C. Qiu, T. Pfrommer, M. Kloft, S. Mandt, and M. Rudolph. Neural transformation learning for deep anomaly detection beyond images. In M. Meila and T. Zhang, editors, *ICML*, volume 139 of *Proceedings of Machine Learning Research*, pages 8703–8714. PMLR, 2021.
- [31] V. Rey, P. M. Sánchez Sánchez, A. Huertas Celdrán, and G. Bovet. Federated learning for malware detection in IoT devices. *Computer Networks*, 204:108693, 2022.
- [32] L. Ruff, J. R. Kauffmann, R. A. Vandermeulen, G. Montavon, W. Samek, M. Kloft, T. G. Dietterich, and K.-R. Müller. A Unifying Review of Deep and Shallow Anomaly Detection. *Proceedings of the IEEE*, 109(5):756–795, 2021.
- [33] L. Ruff, R. Vandermeulen, N. Goernitz, L. Deecke, S. A. Siddiqui, A. Binder, E. Müller, and M. Kloft. Deep one-class classification. In J. Dy and A. Krause, editors, *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 4393–4402. PMLR, 10–15 Jul 2018.
- [34] A. Saeed, T. Ozcelebi, and J. Lukkien. Multi-task self-supervised learning for human activity detection. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, 3(2):1–30, 2019.
- [35] F. Sattler, K.-R. Müller, and W. Samek. Clustered federated learning: Model-agnostic distributed multitask optimization under privacy constraints. *IEEE transactions on neural networks and learning systems*, 32(8):3710–3722, 2020.
- [36] M. Savić, M. Lukic, D. Danilovic, Z. Bodroski, D. Bajović, I. Mezei, D. Vukobratovic, S. Skrbic, and D. Jakovetić. Deep Learning Anomaly Detection for Cellular IoT With Applications in Smart Logistics. *IEEE Access*, 9:59406–59419, 2021.
- [37] M. Savić, J. Atanasijević, D. Jakovetić, and N. Krejić. Tax evasion risk management using a Hybrid Unsupervised Outlier Detection method. *Expert Systems with Applications*, 193:116409, 2022.
- [38] S. Stolfo, W. Fan, W. Lee, A. Prodromidis, and P. Chan. KDD Cup 1999 Data. UCI Machine Learning Repository, 1999. DOI: <https://doi.org/10.24432/C51C7N>.
- [39] M. Tahir, T. Mawla, F. M. Awaysheh, S. Alawadi, M. Gupta, and M. I. Ali. Securefedprom: A zero-trust federated learning approach with multi-criteria client selection. *IEEE J. Sel. Areas Commun.*, 43(6):2025–2041, 2025.
- [40] D. Thakur, A. Guzzo, G. Fortino, and F. Piccialli. Green federated learning: A new era of green aware ai. *ACM Computing Surveys*, 2025.
- [41] B. van Berlo, A. Saeed, and T. Ozcelebi. Towards federated unsupervised representation learning. In *Proceedings of the Third ACM International Workshop on Edge Systems, Analytics and Networking*, EdgeSys '20, page 31–36, New York, NY, USA, 2020. Association for Computing Machinery.
- [42] K. Wang, R. Mathews, C. Kiddon, H. Eichner, F. Beaufays, and D. Ramage. Federated evaluation of on-device personalization. *arXiv preprint arXiv:1910.10252*, 2019.
- [43] X. Wang, Y. Chen, and O. A. Dobre. Federated Learning for Anomaly Detection: A Case of Real-World Energy Storage Deployment. In *ICC 2022 - IEEE International Conference on Communications*, pages 4312–4317, 2022.
- [44] S. Zhai, Y. Cheng, W. Lu, and Z. Zhang. Deep structured energy based models for anomaly detection. In *Proceedings of the 33rd International Conference on Machine Learning - Volume 48*, ICML'16, page 1100–1109. JMLR.org, 2016.
- [45] F. Zhang, K. Kuang, L. Chen, Z. You, T. Shen, J. Xiao, Y. Zhang, C. Wu, F. Wu, Y. Zhuang, and X. Li. Federated unsupervised representation learning. *Frontiers of Information Technology & Electronic Engineering*, 24(8):1181–1193, 2023.